Web Technology Practicals – Practical 6A & 6B

Combined practical instructions transcribed from the provided images.

Practical 6A: College Information Portal as a Progressive Web Application (PWA)

Objective: Create a simple website for a college information portal and convert it into a progressive web application.

1. Create an ASP.NET Core Web App.

Search for “ASP.NET Core Web App (Model-View-Controller)” and click Next.

Project Name: CollegePortalPWA

Framework: .NET 10 or the latest available version. Click Create.

2. Update the Home Page.

Open Views → Home → Index.cshtml and replace the existing HTML with:

```
<div class="text-center">
    <h1>Vartak College Information Portal</h1>
    <h3>Welcome to Vartak College</h3>
    <p>Courses Offered:</p>
    <ul>
        <li>BSC IT</li>
        <li>BSC CS</li>
        <li>BSC DS</li>
    </ul>
    <p>Contact: 9818763456</p>
</div>
```

3. Add PWA support – Service Worker.

Right-click wwwroot → Add → New Item → JavaScript File. Name it service-worker.js.

```
self.addEventListener('install', function (event) {
    console.log('Service Worker Installed');
});
self.addEventListener('fetch', function (event) {
    console.log('Fetching:', event.request.url);
});
```

4. Create the Manifest File.

Right-click wwwroot → Add → New Item → JSON File. Name it manifest.json.

```
{
  "name": "Vartak College Information Portal",
  "short_name": "CollegePortal",
  "start_url": "/",
  "display": "standalone",
  "background_color": "white",
  "theme_color": "blue",
  "icons": [
    {
      "src": "/icons/icon1.png",
      "sizes": "192x192",
      "type": "image/png"
    },
    {
      "src": "/icons/icon2.png",
```

```
      "sizes": "512x512",
      "type": "image/png"
    }
  ]
}
```

5. Create an icons folder.

Right-click wwwroot → Add → New Folder. Name it icons. Add icon1.png and icon2.png inside it.

6. Register the manifest.

Open Views → Shared → _Layout.cshtml. Add this tag inside the <head> section:

```
<link rel="manifest" href="~/manifest.json">
```

7. Register the Service Worker.

In the same _Layout.cshtml file, add the following script before </body>:

```
<script>
if ('serviceWorker' in navigator) {
    navigator.serviceWorker.register('/service-worker.js')
        .then((registration) => {
            console.log('Service Worker Registered!', registration);
        })
        .catch((error) => {
            console.log('Service Worker Registration Failed:', error);
        });
}
</script>
```

8. Verify that static files are enabled.

In Program.cs, ensure this line exists; otherwise, add it in the appropriate place:

```
app.UseStaticFiles();
```

9. Run the application.

Open the application in Chrome and click the PWA Install icon in the address bar to install the app.

10. Verify the Service Worker.

Press F12 → Application → Service Workers. Check that service-worker.js is activated and running.

Practical 6B: Create a PWA that Displays a Custom Offline Page

Objective: Create a PWA that displays a custom offline page when the user is not connected to the internet.

1. Create an ASP.NET Core MVC project.

Project Name: OfflinePWA

2. Select the framework.

Choose .NET 10 or the latest available version and click Create.

3. Create the Home Page.

Open Views → Home → Index.cshtml and add:

```
<div class="text-center">
    <h1>VARTAK College Information Portal</h1>
    <h3>Welcome to VARTAK College</h3>
    <p>This website supports Offline Mode.</p>
</div>
```

4. Create the Offline Page.

Right-click wwwroot → Add → New Item → HTML Page. Name it offline.html.

```
<!DOCTYPE html>
<html>
<head>
    <title>Offline</title>
</head>
<body>
    <h1>No Internet Connection</h1>
    <h3>Please check your internet connection.</h3>
</body>
</html>
```

5. Create the Manifest File.

Right-click wwwroot → Add → New Item → JSON File. Name it manifest.json.

```
{
  "name": "Offline PWA",
  "short_name": "OfflinePWA",
  "start_url": "/",
  "display": "standalone",
  "background_color": "white",
  "theme_color": "blue",
  "icons": [
    {
      "src": "/icons/icon1.png",
      "sizes": "192x192",
      "type": "image/png"
    },
    {
      "src": "/icons/icon2.png",
      "sizes": "512x512",
      "type": "image/png"
    }
  ]
}
```

6. Create the icons folder.

Right-click wwwroot → Add → New Folder. Name it icons. Add icon1.png and icon2.png inside it.

7. Create the Service Worker for offline caching.

Right-click wwwroot → Add → New Item → JavaScript File. Name it service-worker.js.

```
const CACHE_NAME = "offline-cache-v1";
const urlsToCache = ["/", "/offline.html"];

self.addEventListener("install", event => {
    event.waitUntil(
        caches.open(CACHE_NAME).then(cache => {
            return cache.addAll(urlsToCache);
        })
    );
});

self.addEventListener("fetch", event => {
    event.respondWith(
        fetch(event.request).catch(() => {
            return caches.match("/offline.html");
        })
    );
});
```

8. Register the Manifest.

Open Views → Shared → _Layout.cshtml and add this link tag inside <head>:

```
<link rel="manifest" href="~/manifest.json">
```

9. Register the Service Worker.

In the same _Layout.cshtml file, add the following script before </body>:

```
<script>
if ('serviceWorker' in navigator) {
    navigator.serviceWorker.register('/service-worker.js')
        .then(() => console.log('Service Worker Registered'));
}
</script>
```

10. Verify static files.

In Program.cs, ensure the following line exists:

```
app.UseStaticFiles();
```

11. Run the application.

12. Verify the Service Worker.

Press F12 → Application → Service Workers and verify the status.

13. Test the Offline Page.

Press F12 → Network tab → select Offline → refresh the page.

Note: The source images show the practical instructions. Icon image files (icon1.png and icon2.png) must be supplied or created separately for the manifest icons to display correctly.